

고급머신러닝 보고서

RNN

Recurrent Neural Network

전자공학부 전자공학과
2020144005 김윤성
2024. 11. 12 제출

목차 INDEX

01 RNN

- RNN
- LSTM
- GRU
- Seq2Seq
- Attention
- Transformer

02 실습 #1)

- RNN, LSTM 최적화
- 코드 분석
- 실행 결과

03 실습 #2)

- 한계점 개선
- 코드 분석
- 실행 결과
- 실행 비교
- 결과 분석

The background is a solid dark blue with several geometric shapes. A large, light blue curved shape is in the upper right. A dark blue diagonal band runs from the bottom left towards the top right. A lighter blue diagonal band runs from the top left towards the bottom right.

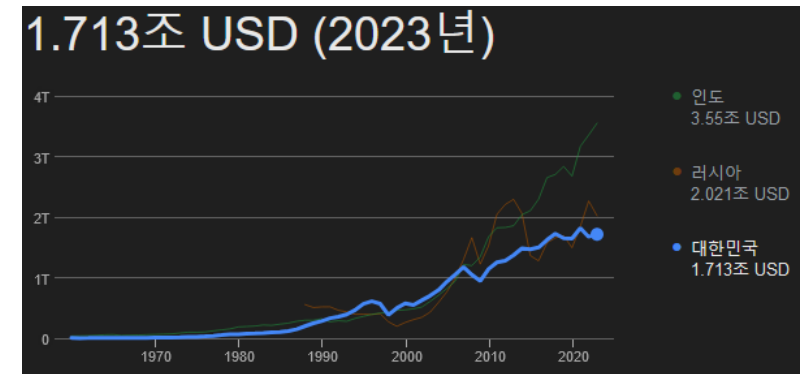
01

RNN

Recurrent Neural Network

시계열 데이터 Time Series Data

- 일정한 시간 간격을 두고 순서대로 기록된 데이터로, 시간의 흐름에 따라 변화하는 값을 나타냄
- 주식 가격, 날씨 변화, 판매량, 경제 지표 등이 시계열 데이터의 예시
- 과거의 패턴을 분석해 미래를 예측하거나 트렌드를 파악하는 데 유용
- Autocorrelation, Trend, Seasonality, Volatility 등의 특징이 있음
- 시계열 데이터 분석에서는 Moving Average, ARIMA (Autoregressive Integrated Moving Average)와 같은 통계적 기법과 RNN, LSTM 등의 딥러닝 기법이 많이 사용됨



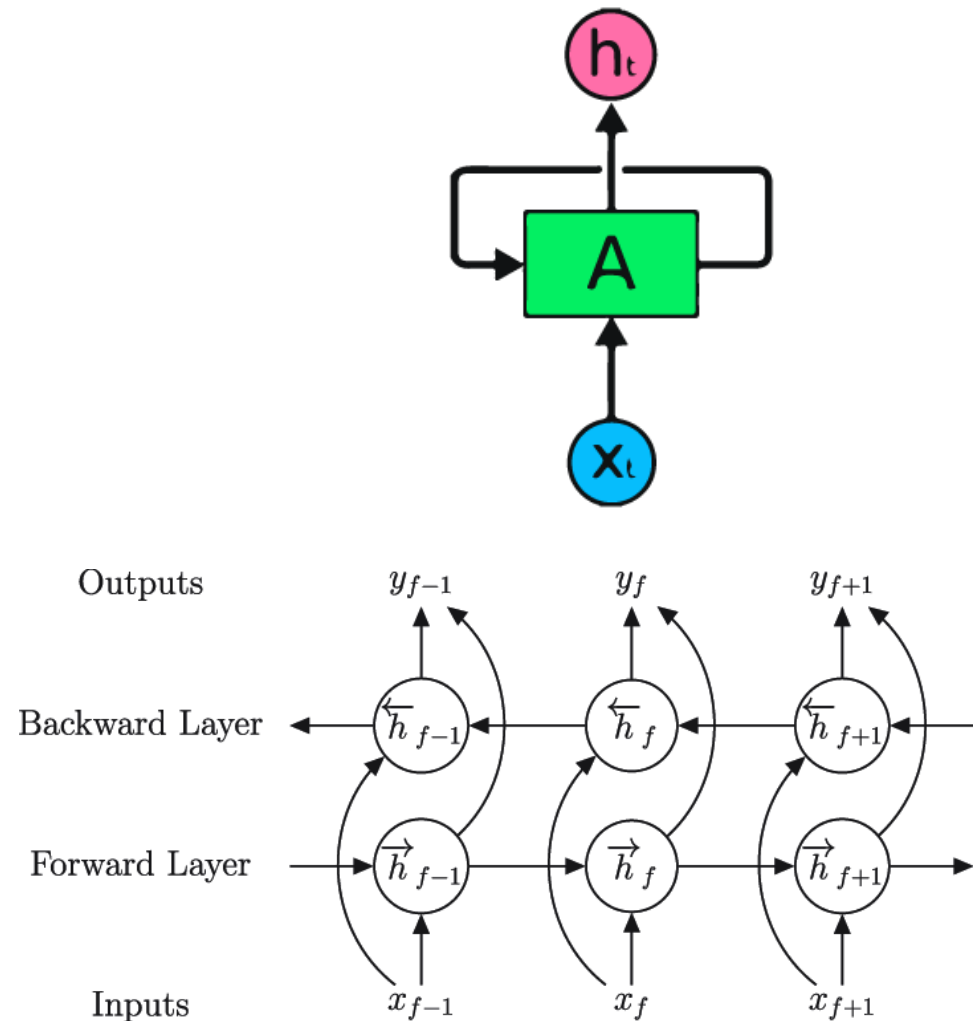
GDP 성장 데이터



주가 데이터

RNN (Vanilla) Recurrent Neural Network

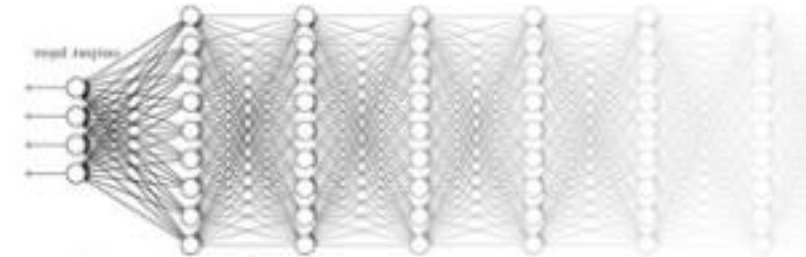
- 순환 신경망으로, 시퀀스 데이터의 순서와 시간적인 의존성을 반영하여 정보를 처리하는 인공신경망
- 이전 단계의 출력이 현재 단계의 입력으로 연결되는 순환 구조를 통해 연속적인 데이터를 처리할 수 있음
- 자연어 처리, 음성 인식, 시계열 데이터 분석 등의 연속적인 패턴을 갖는 데이터에 적합
- 시계열 데이터를 반대 방향으로 다시 한 번 사용하는 변종이 있는데, Bi-Directional RNN이라 함. 이는 뒤의 LSTM, GRU 등에도 적용 가능



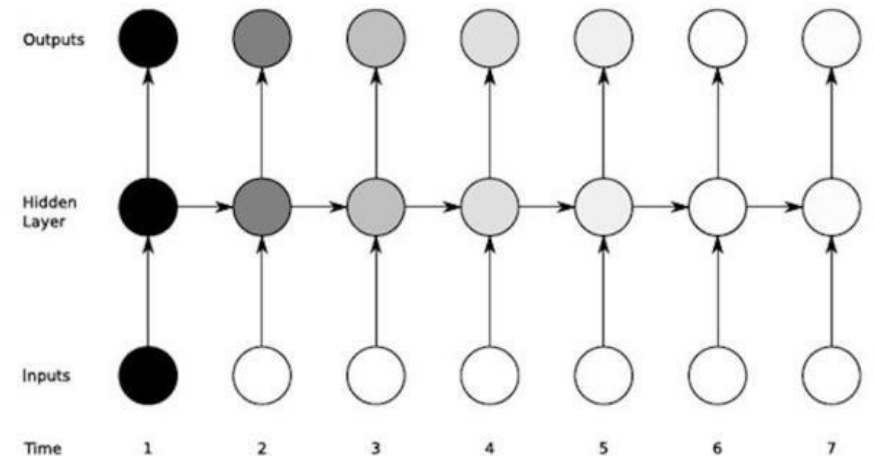
Bidir RNN의 구조

RNN의 단점 Recurrent Neural Network

- RNN은 긴 시계열 데이터를 다룰 때 기울기가 급격히 작아져 학습이 잘 되지 않는 문제가 발생 => ReLU와 같은 함수를 쓰더라도 구조적 특성상 Gradient Exploding 문제 발생 가능
- 이전 단계의 정보가 멀리 떨어진 시점까지 전달되지 못해, 긴 시퀀스 데이터의 장기적인 패턴을 효과적으로 학습하는데 제한이 있음
- RNN은 각 단계가 순차적으로 연결되므로 병렬 처리가 어렵고, 이는 학습 속도의 저하를 불러옴

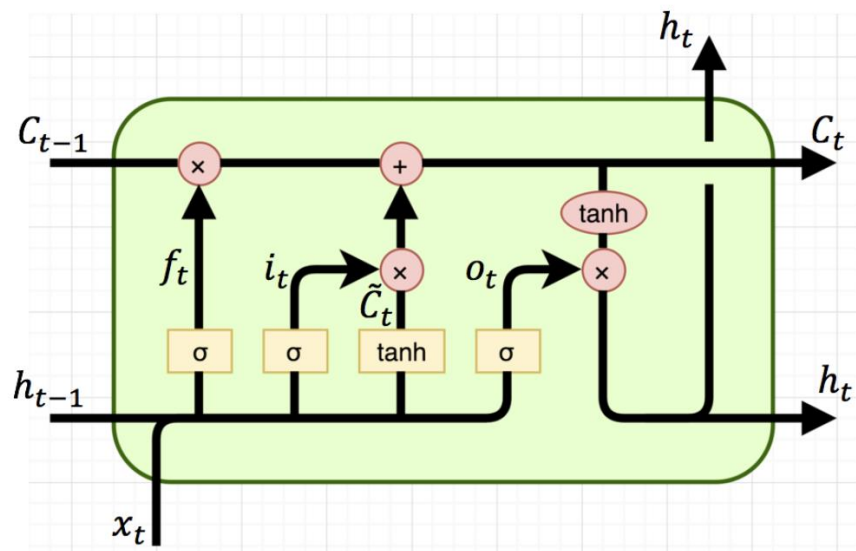


Gradient Vanishing Problem
(그림은 DNN)



LSTM Long Short-Term Memory

- RNN에서 발생하는 긴 시계열 데이터에서의 Gradient Vanishing 문제를 해결하기 위해 개발됨
- Cell State와 Gate 의 집합으로 구성됨
- Input Gate (i_t): 새로운 정보를 셀 상태에 얼마나 반영할지 결정
- Forget Gate (f_t): 과거 정보를 얼마나 유지할지 선택하여 필요 없는 정보를 삭제
- Output Gate (o_t): 현재 시점의 출력을 결정
- 시퀀스 데이터의 장기적인 의존성을 더 잘 학습하고, 중요하지 않은 정보를 점진적으로 잊어버릴 수 있음
- [논문 링크](#)



LSTM의 구조

LSTM Long Short-Term Memory

- $f_t = \sigma(W_f \cdot [h_{t-1}, x_t] + b_f)$

f_t : Forget Gate의 값
 h_{t-1} : 이전 시점의 Hidden State
 σ : Sigmoid 함수
 W_f : 각 게이트에 해당하는 Weight

- $i_t = \sigma(W_i \cdot [h_{t-1}, x_t] + b_i)$
- $\tilde{C}_t = \tanh(W_C \cdot [h_{t-1}, x_t] + b_c)$

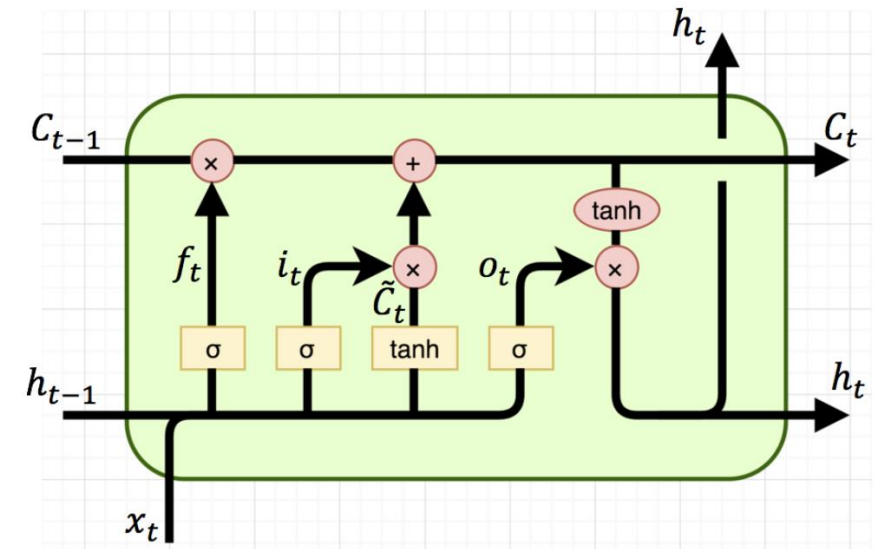
h_{t-1} : 이전 시점의 Hidden State
 σ : Sigmoid 함수
 W_i, W_C : Weight

- $C_t = f_t \odot C_{t-1} + i_t \odot \tilde{C}$

Hadamard Product

- $O_t = \sigma(W_O \cdot [h_{t-1}, x_t] + b_o)$
- $h_t = O_t \odot \tanh C_t$

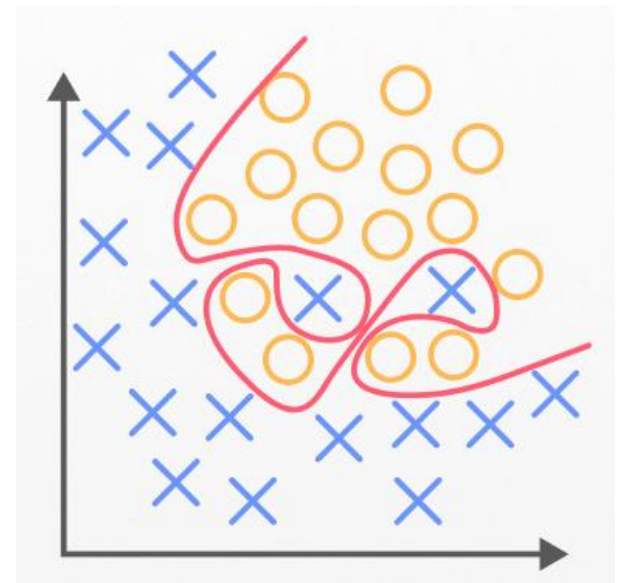
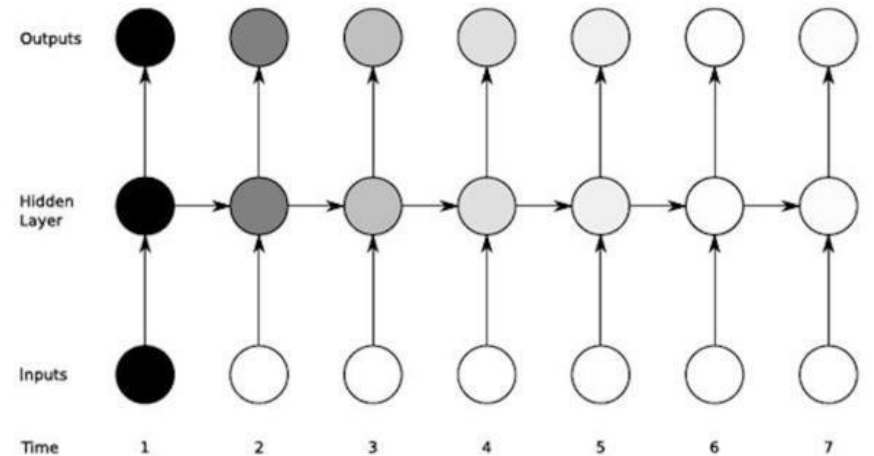
h_{t-1} : 이전 시점의 Hidden State
 h_t : 현재 시점의 Hidden State
 W_o : Weight



LSTM의 구조

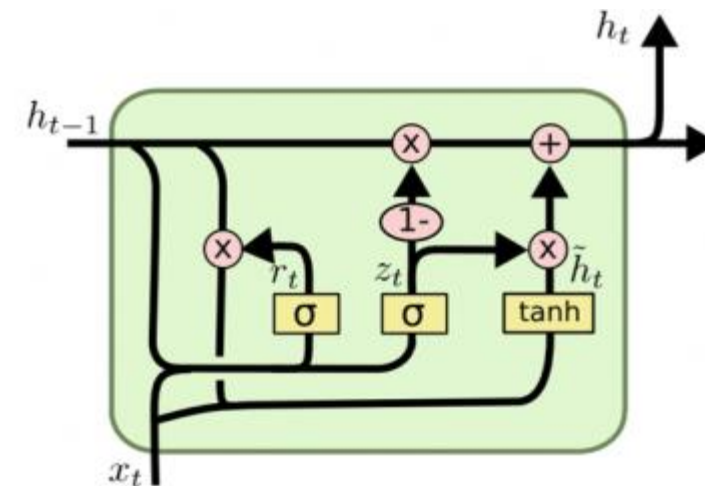
LSTM의 단점 Long Short-Term Memory

- LSTM은 기본 RNN보다 복잡한 셀 구조와 여러 게이트로 이루어져 있어 계산 비용이 높고 학습 시간이 오래 걸림
- RNN과 같이 시퀀스 데이터를 순차적으로 처리해야 하므로 병렬화가 어려워 학습 속도가 느려질 수 있고, 이는 특히 긴 시퀀스 데이터에서는 효율성을 저하시킴
- 모델이 복잡하다 보니, 데이터가 충분하지 않거나 모델이 너무 깊을 경우 쉽게 Overfitting될 위험이 있음
- LSTM은 RNN에 비해 장기 의존성을 잘 학습할 수 있지만, 여전히 매우 긴 시계열 데이터에서는 한계가 있음



GRU Gated Recurrent Unit

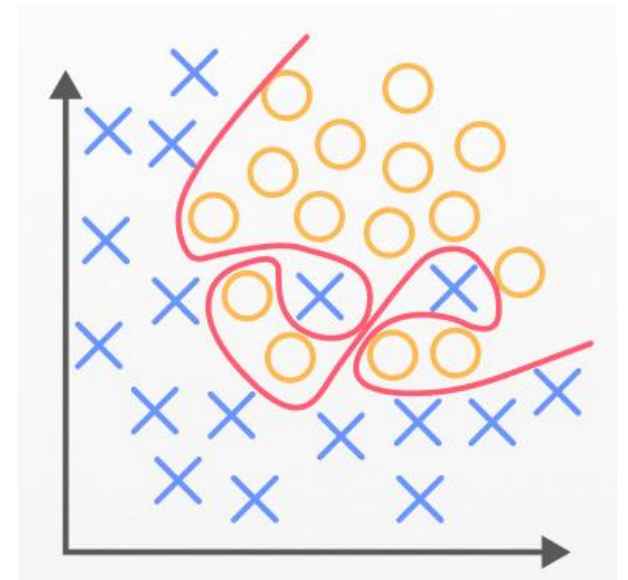
- LSTM과 유사하지만 두 개의 게이트만 사용해 구조가 더 단순
- 업데이트 게이트 (z_t): 이전 상태에서 현재 상태로 얼마나 정보를 유지할지를 결정
- 리셋 게이트 (r_t): 이전 정보를 얼마나 새로운 입력과 결합할지를 조정하여 과거 정보를 부분적으로 초기화
- GRU는 LSTM보다 적은 메모리와 연산을 요구하면서도, LSTM 대비 근접하거나 때로는 더 나은 결과를 보일 수 있음
- 실시간 응용이나 모바일 환경처럼 빠른 연산이 요구되는 상황에서 많이 사용됨



GRU의 구조

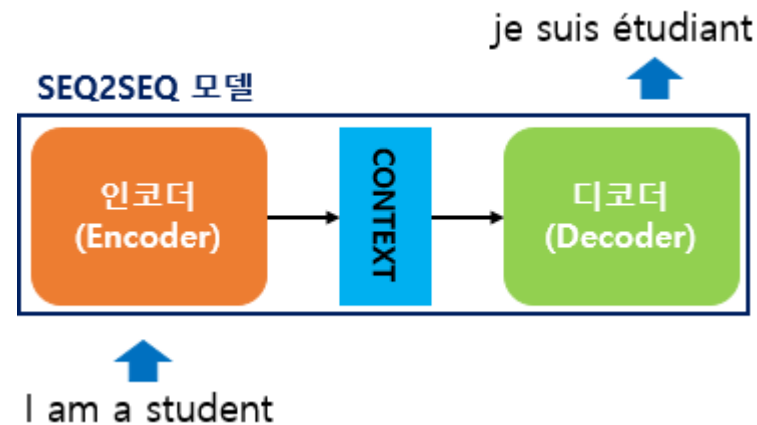
GRU의 단점 Gated Recurrent Unit

- RNN과 같이 시퀀스 데이터를 순차적으로 처리해야 하므로 병렬화가 어려워 학습 속도가 느려질 수 있고, 이는 특히 긴 시퀀스 데이터에서는 효율성을 저하시킴
- LSTM보다 연산량은 적으나, 복잡한 의존 관계가 많은 데이터에서는 정보 소실 발생 가능
- GRU의 성능은 Hyperparameter에 크게 좌우되기 때문에 최적화가 어렵고 Overfitting의 위험이 높음

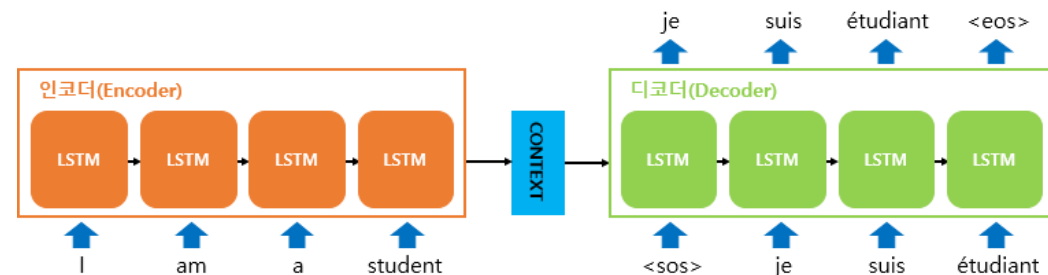


Seq 2 Seq Sequence-to-Sequence

- 입력 시퀀스를 다른 형태의 출력 시퀀스로 변환하는 모델
- 일반적으로 두 개의 RNN(혹은 LSTM, GRU) 네트워크로 구성된 인코더-디코더 구조
- Encoder: 입력 시퀀스를 받아 하나의 고정된 벡터(Context Vector)로 압축, 이 벡터는 입력 시퀀스의 의미를 요약하여 디코더로 전달됨
- Decoder: 인코더의 출력을 받아, 이를 바탕으로 목표 출력 시퀀스 생성. 이 과정에서 이전 시점의 출력이 다음 시점의 입력으로 사용됨



Seq2Seq 의 구조



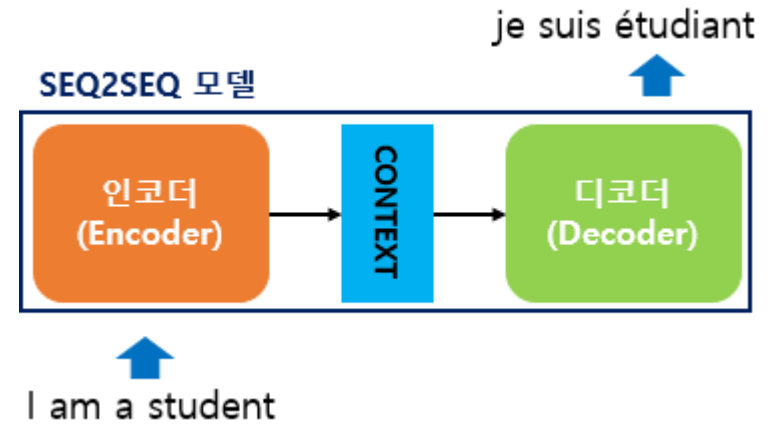
Seq2Seq의 Encoder, Decoder의 구조

출처

Seq 2 Seq의 단점 Sequence-to-Sequence

- 하나의 고정된 크기의 벡터 (Context Vector)에 정보를 때려박음
=> 정보 손실 위험
- RNN 구조의 특성상 Gradient Vanishing Problem 발생 가능
- 기존 모델들과 동일한 문제 (병렬 학습 불가,
긴 시퀀스 학습 시 과거의 정보를 효과적으로 사용하지 못함)

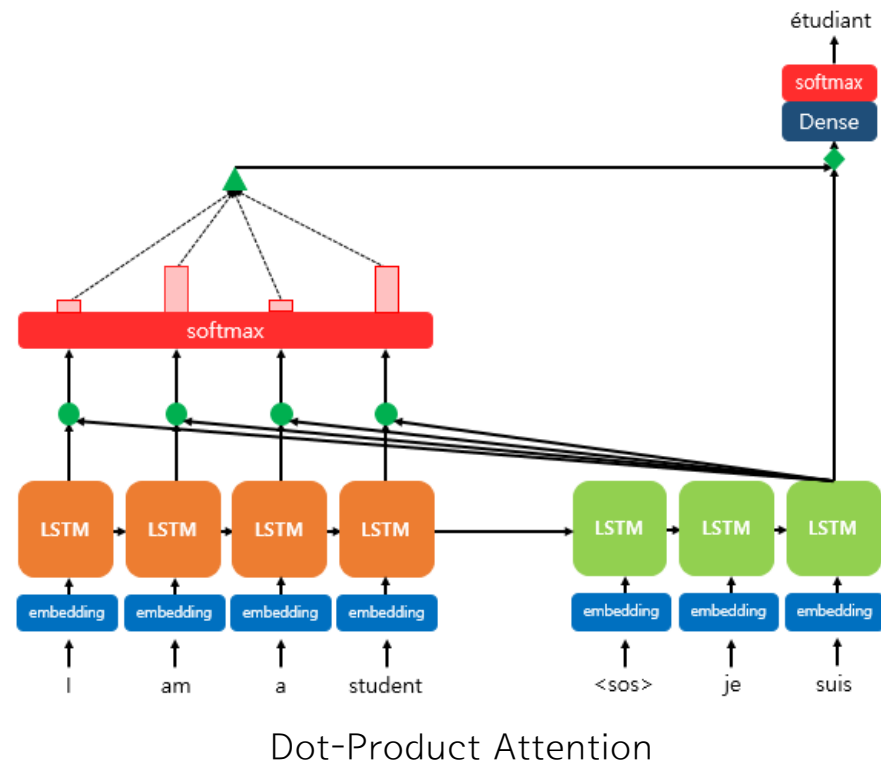
➡ NLP(자연어처리) 성능 하락



정보 손실

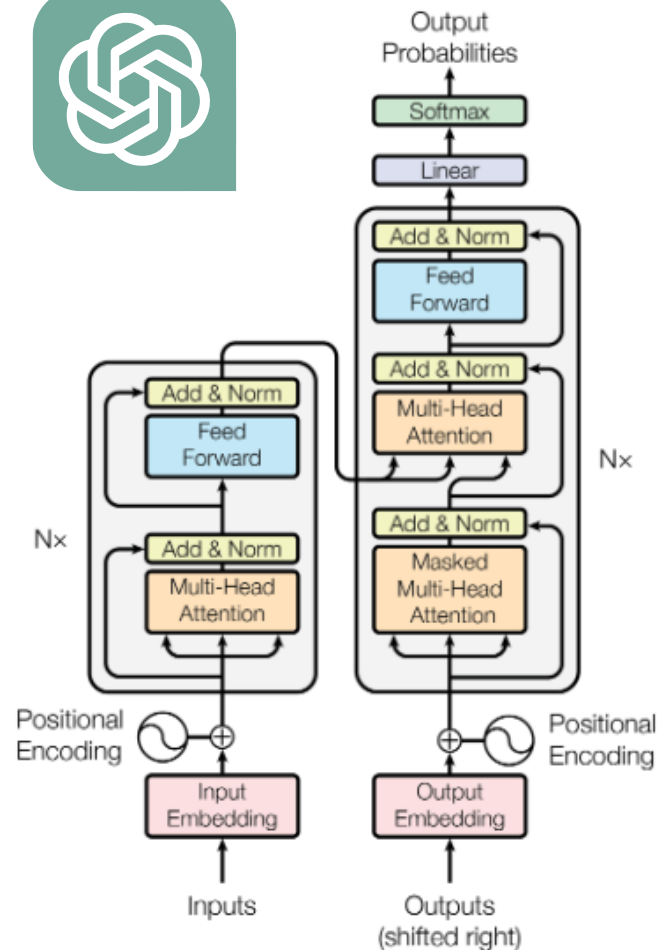
Attention Attention Mechanism

- 디코더에서 출력 단어를 예측하는 매 시점(time step)마다, 인코더에서의 전체 입력 문장을 다시 한 번 참고
- 단, 전체 입력 문장을 전부 다 동일한 비율로 참고하는 것이 아니라, 해당 시점에서 예측해야 할 단어와 연관이 있는 입력 단어 부분을 좀 더 집중해서 봄
- 1. 입력 시퀀스의 각 요소에 대해 중요도 가중치를 계산 (이 가중치는 현재 상태와 시퀀스의 각 요소 간의 유사성을 측정해 계산됨)
- 2. 계산된 가중치를 기반으로, 각 요소의 정보를 가중합하여 Context Vector를 생성 (중요한 정보에 더 많은 가중치를 주어 중요한 부분이 더 강조됨)
- 3. 생성된 컨텍스트 벡터를 디코더나 다른 레이어의 입력으로 사용하여, 시퀀스의 중요 정보를 반영한 출력을 만듦



Transformer

- 2017년에 발표된 모델이며, RNN구조와 CNN을 사용하지 않고도 시계열 데이터에 대한 병렬 학습 가능
- Self-Attention 입력 시퀀스 내의 모든 단어가 서로를 참조해 각 단어의 중요도를 계산하므로 인해 긴 시퀀스에서도 각 단어 간의 관계를 효과적으로 학습할 수 있음
- Transformer는 인코더와 디코더로 구성되는데, 인코더는 입력 시퀀스를 처리해 요약된 표현을 만들고, 디코더는 이를 바탕으로 예측 결과를 생성
- 순환 구조가 없기 때문에 모든 시퀀스를 동시에 처리할 수 있어 학습 속도가 빠르며, 특히 긴 시퀀스에서도 이전 구조들과 다르게 문제가 적음
- [논문 링크](#) (Attention Is All You Need) 추후 논문 분석 예정



Transformer의 구조

02

실습 #1

Optimize RNN & LSTM

실습 #1)

앞선 실습에서 진행한 RNN 및 LSTM의 학습 파라미터 최적화

- yahoo finance를 활용하여 데이터 수집 (원하는 종목 코드를 골라 사용해도 무관)
- 자신만의 신경망 모델 디자인 (계층 추가, 활성화 함수 등등)
- 여러 파라미터 조절에 따른, 결과 비교 분석 (Learning rate, Cell 개수, Epoch, Window Size, Optimizer, Loss 등등)
- RNN 및 LSTM에 대한 Train set 및 Test set의 Loss값 및 예측 결과 비교 분석
- 조건 : Train set 및 Test set 기간의 합은 최소 4년이 넘도록 할 것

02 실습 #1 코드 분석

```
import yfinance as yf
import matplotlib.pyplot as plt
import numpy as np
import keras
```

필요한 라이브러리를 임포트한다.

```
STOCK_NAME = "TSLA" # 테슬라
STOCK_TRAIN = yf.download(STOCK_NAME, end='2020-12-31')
STOCK_TEST = yf.download(STOCK_NAME, start='2021-01-01')
```

분석을 위한 종목은 테슬라, Training Set은 2020년 12월 31일까지의 데이터를, Test Set은 2021년 1월 1일 이후의 데이터를 사용했다.

```
STOCK_Close_training = np.array(STOCK_TRAIN['Close'])
STOCK_Close_testing = np.array(STOCK_TEST['Close'])
date_training = STOCK_TRAIN.index.date
date_testing = STOCK_TEST.index.date

window_size = 1
x_train, t_train = [], []
x_test, t_test = [], []

for i in range(len(STOCK_Close_training) - window_size):
    x_train.append(STOCK_Close_training[i: i + window_size])
    t_train.append(STOCK_Close_training[i + window_size])

for i in range(len(STOCK_Close_testing) - window_size):
    x_test.append(STOCK_Close_testing[i: i + window_size])
    t_test.append(STOCK_Close_testing[i + window_size])
```

Training Set과 Test Set 모두 종가만을 기준으로 분석하였는데, 종가를 기반으로 다음날의 종가를 예측하는 것이 (추가된) 목표이기 때문이다. X_에는 window size만큼의 데이터(1일)을 저장하고, t_에는 window size 다음날의 정보를 저장한다.

02 실습 #1 코드 분석

```
max_value = np.max(STOCK_Close_training)
x_train = np.array(x_train) / max_value
t_train = np.array(t_train) / max_value
x_test = np.array(x_test) / max_value
t_test = np.array(t_test) / max_value

x_train = x_train.reshape((x_train.shape[0], window_size, 1))
x_test = x_test.reshape((x_test.shape[0], window_size, 1))
```

Training Set과 Test Set을 정규화한다.
정규화하는 이유는 모델의 학습을 더 쉽게 하고, 수렴을 빠르게 하기 위함이다.

```
RNN = keras.models.Sequential()
RNN.add(keras.layers.SimpleRNN(units=128, activation='leaky_relu', input_shape=(window_size, 1)))
RNN.add(keras.layers.Dense(1, activation='linear'))
RNN.compile(optimizer='nadam', loss='MSE')

LSTM = keras.models.Sequential()
LSTM.add(keras.layers.LSTM(units=128, activation='leaky_relu', input_shape=(window_size, 1)))
LSTM.add(keras.layers.Dense(1, activation='linear'))
LSTM.compile(optimizer='adamax', loss='MSE')
```

RNN 모델과 LSTM 모델의 구성이다.
이 파라미터들은 Brute Force Search 방식으로 최적의 파라미터를 찾았는데, 그 방법은 추후 소개한다.

02 실습 #1 코드 분석

```
early_stopping = keras.callbacks.EarlyStopping(monitor='loss', patience=30)

print("<RNN Start.....>")
history_rnn = RNN.fit(x_train, t_train, epochs=500, verbose=1, callbacks=early_stopping)
print("RNN Complete!\n")
print("<LSTM Start.....>")
history_lstm = LSTM.fit(x_train, t_train, epochs=500, verbose=1, callbacks=early_stopping)
print("LSTM Complete!\n")
```

모델을 학습시키는 부분이며, 빠른 학습을 위하여 Early Stopping을 적용하였고, patience 값은 30으로 설정하였다.

```
pred_test = RNN.predict(x_test)
lstm_pred_test = LSTM.predict(x_test)
pred_test = pred_test * max_value
lstm_pred_test = lstm_pred_test * max_value
```

각 모델에 대해 Test Set으로 예측을 한다.

현재 값은 정규화된 값이므로 max_value를 곱하여 다시 실제 값으로 변환해준다.

02 실습 #1 코드 분석

```
recent_data = (STOCK_Close_testing[-window_size:] / max_value).reshape(1, window_size, 1)

next_day_rnn = RNN.predict(recent_data)
next_day_lstm = LSTM.predict(recent_data)

next_day_rnn = next_day_rnn[0, 0] * max_value
next_day_lstm = next_day_lstm[0, 0] * max_value
```

코드에서 새로 추가된 목표인 다음날 종가를 예측하기 위한 코드이다.
Test Set의 마지막 window size일 만큼의 데이터로 그 다음날 (내일)의 종가를 예측한다.

```
print("=====")
print(f"(RNN) Predicted next day closing price: {float(next_day_rnn):.2f}$")
print(f"(LSTM) Predicted next day closing price: {float(next_day_lstm):.2f}$")
print("=====")

plt.figure(1)
plt.plot(date_testing[window_size:], pred_test, label='RNN', color='red', alpha=0.6)
plt.plot(date_testing[window_size:], lstm_pred_test, label='LSTM', color='green', alpha=0.7)
plt.plot(date_testing, STOCK_Close_testing, label=f"{STOCK_NAME}'s Adjusted Close", color='blue', linewidth=2)

plt.xlabel("Date")
plt.ylabel("USD ($)")
plt.title(f"{STOCK_NAME}'s Close (2021~)")
plt.grid()
plt.legend()
plt.tight_layout()

plt.figure(figsize=(15, 10))
plt.plot(history_rnn.history['loss'], label='RNN', color='red')
plt.plot(history_lstm.history['loss'], label='LSTM', color='green')
plt.plot(history_gru.history['loss'], label='GRU', color='gold')
plt.plot(history_bidir.history['loss'], label='Bi-Directional RNN', color='cyan')

plt.xlabel('Epochs')
plt.ylabel('Loss')
plt.xscale('log')
plt.title('Training Loss per Model')
plt.legend()
plt.grid()
plt.tight_layout()

plt.show()
```

최종적으로 다음날의 예상 종가와 Loss, 예측된 종가 그래프를 그리는 부분이다.

02 실습 #1 코드 분석

```
import yfinance as yf
import numpy as np
import keras
import tensorflow as tf
```

여기서부터는 Brute Force 방식으로 최적의 파라미터를 찾는 코드이다.
필요한 라이브러리들을 임포트한다.

```
STOCK_NAME = "TSLA"
STOCK_training = yf.download(STOCK_NAME, end='2020-12-31')
STOCK_test = yf.download(STOCK_NAME, start='2021-01-01')

STOCK_training_close = np.array(STOCK_training['Close'])
Date = STOCK_training.index.date
Xtick = np.arange(0, date.shape[0], int(date.shape[0] / 3))

STOCK_test_close = np.array(STOCK_test['Close'])
Date_test = STOCK_test.index.date

def prepare_data(window_size):
    x_train, t_train, x_test, t_test = [], [], [], []
    for I in range(len(STOCK_training_close) - window_size):
        x_train.append(STOCK_training_close[i:i+window_size])
        t_train.append(STOCK_training_close[i+window_size])
    x_train = np.array(x_train)/np.max(x_train)
    t_train = np.array(t_train)/np.max(t_train)

    for I in range(len(STOCK_test_close) - window_size):
        x_test.append(STOCK_test_close[i:i+window_size])
        t_test.append(STOCK_test_close[i+window_size])
    x_test = np.array(x_test)/np.max(x_test)
    t_test = np.array(t_test)/np.max(t_test)
    return x_train, t_train, x_test, t_test
```

위 코드와 동일하게 Training Set과 Test Set을 생성하는 부분이나, 이 코드에서는 함수로 정의하여 사용하였다.

02 실습 #1 코드 분석

```
# Define parameters to test
optimizers = ['adam', 'adamax', 'nadam']
activations = ['relu', 'tanh', 'selu', 'leaky_relu']
window_sizes = [1, 2, 3]
num_cells_options = [1, 2, 3, 4, 5]

# 모델 당 총 학습 수
total_training_iteration = len(optimizers) * len(activations) * len(window_sizes) * len(num_cells_options)

rnn_loss = {}
lstm_loss = {}
```

옵티마이저와 활성화함수를 선택한다.

몇 번의 실행 결과, 최고의 결과는 대부분 해당 파라미터들에서 나왔기에 이렇게 설정하였다.

Total_training_iteration은 각 모델 당 총 학습 수이며, 학습 중 어디까지 학습되었는지 알기 위해 추가한 변수이다.

```
def get_optimizer(optim):
    optimizer = tf.keras.optimizers.get(optim)
    return optimizer
```

위에서 선언된 optimizers 리스트의 옵티마이저를 하나씩 keras에서 가져오는 함수이다.

02 실습 #1 코드 분석

```
def build_model(model_type, num_cells, activation, window_size):
    model = keras.models.Sequential()
    for i in range(num_cells):
        return_sequences = bool(i < num_cells - 1)
        if activation == 'leaky_relu':
            if model_type == 'RNN':
                model.add(keras.layers.SimpleRNN(128, activation=keras.layers.LeakyReLU(alpha=0.01), return_sequences=return_sequences,
input_shape=(window_size, 1)))
            elif model_type == 'LSTM':
                model.add(keras.layers.LSTM(128, activation=keras.layers.LeakyReLU(alpha=0.01), return_sequences=return_sequences,
input_shape=(window_size, 1)))
            else:
                if model_type == 'RNN':
                    model.add(keras.layers.SimpleRNN(128, activation=activation, return_sequences=return_sequences, input_shape=(window_size,
1)))
                elif model_type == 'LSTM':
                    model.add(keras.layers.LSTM(128, activation=activation, return_sequences=return_sequences, input_shape=(window_size, 1)))
    model.add(keras.layers.Dense(1, activation=activation if activation != 'leaky_relu' else keras.layers.LeakyReLU(alpha=0.01)))
    return model
```

(바닐라)RNN과 LSTM 구조를 build 하는 함수이다.

가변적인 파라미터가 상당히 많은데, 모델의 종류, 셀의 개수, 활성화 함수의 종류, window size의 크기를 받아 해당 파라미터들의 값대로 모델을 구성한다.

02 실습 #1 코드 분석

```
best_rnn_config = None
best_rnn_loss = float('inf')
best_rnn_model = None
best_rnn_predictions = None
training_counter = 1
for optim in optimizers:
    for activation in activations:
        for window_size in window_sizes:
            x_train, t_train, x_test, t_test = prepare_data(window_size)
            for num_cells in num_cells_options:
                optimizer = get_optimizer(optim)
                model = build_model('RNN', num_cells, activation, window_size)
                model.compile(optimizer=optimizer, loss='MSE')
                # Train the model
                early_stopping = keras.callbacks.EarlyStopping(monitor='loss', patience=10, restore_best_weights=True)
                history = model.fit(x_train, t_train, epochs=200, verbose=0, callbacks=[early_stopping])
                stopped_epoch = len(history.history['loss'])
                loss = history.history['loss'][-1]
                config = (optim, activation, window_size, num_cells, stopped_epoch)
                rnn_loss[config] = loss
                print(f"{training_counter} / {total_training_iteration}")
                print(f'Model: RNN Optimizer: {optim} Activation Function: {activation} Window Size: {window_size} Num Cells: {num_cells} Epochs: {stopped_epoch} LOSS: {loss}')
                training_counter += 1
                # Evaluate and save the best model
                if loss < best_rnn_loss:
                    best_rnn_loss = loss
                    best_rnn_config = config
                    best_rnn_model = model
                    best_rnn_predictions = model.predict(x_test).flatten() * np.max(STOCK_training_close)
pred_test = best_rnn_predictions
```

Vanilla RNN구조를 구성하는 코드이다.

앞서 정의된 `get_optimizer()`와 `build_model()`을 이용하여 모든 경우의 수를 각각 계산하게 하였고, Early Stopping 또한 적용하여 불필요한 반복을 최소화했다. 또한, `loss`를 계산하여 `loss`순서대로 생성된 RNN모델들을 정렬하였다.

코드를 살펴보면 가능한 경우의 수들을 모두 전수조사하는 것을 알 수 있는데, 이렇게 모든 경우의 수를 조사하는 것을 Brute Force라 한다.

02 실습 #1 코드 분석

```
best_lstm_config = None
best_lstm_loss = float('inf')
best_lstm_model = None
best_lstm_predictions = None
training_counter = 1
for optim in optimizers:
    for activation in activations:
        for window_size in window_sizes:
            x_train, t_train, x_test, t_test = prepare_data(window_size)
            for num_cells in num_cells_options:
                optimizer = get_optimizer(optim)
                model = build_model('LSTM', num_cells, activation, window_size)
                model.compile(optimizer=optimizer, loss='MSE')
                # Train the model
                early_stopping = keras.callbacks.EarlyStopping(monitor='loss', patience=10, restore_best_weights=True)
                history = model.fit(x_train, t_train, epochs=200, verbose=0, callbacks=[early_stopping])
                stopped_epoch = len(history.history['loss'])
                loss = history.history['loss'][-1]
                config = (optim, activation, window_size, num_cells, stopped_epoch)
                lstm_loss[config] = loss
                print(f"{training_counter} / {total_training_iteration}")
                print(f'Model: LSTM Optimizer: {optim} Activation Function: {activation} Window Size: {window_size} Num Cells: {num_cells} Epochs: {stopped_epoch} LOSS: {loss}')
                training_counter += 1
                # Evaluate and save the best model
                if loss < best_lstm_loss:
                    best_lstm_loss = loss
                    best_lstm_config = config
                    best_lstm_model = model
                    best_lstm_predictions = model.predict(x_test).flatten() * np.max(STOCK_training_close)
lstm_pred_test = best_lstm_predictions
```

앞서 살펴본 최적의 RNN 찾기 알고리즘의 LSTM 버전이다.

02 실습 #1 코드 분석

```
# Get the top 5 best combinations based on loss for both RNN and LSTM
sorted_rnn_losses = sorted(rnn_loss.items(), key=lambda x: x[1][:5])
sorted_lstm_losses = sorted(lstm_loss.items(), key=lambda x: x[1][:5])

average_epoch_rnn = np.mean([config[4] for config, _ in sorted_rnn_losses])
average_epoch_lstm = np.mean([config[4] for config, _ in sorted_lstm_losses])

print("\n\n\n===== Top 5 RNN Configurations with Lowest Loss =====")
for config, loss in sorted_rnn_losses:
    print(f"Optimizer: {config[0]}, Activation: {config[1]}, Window Size: {config[2]}, Num Cells: {config[3]}, Average Epochs: {average_epoch_rnn}, Loss: {loss}")
print("\n\n\n")
print("===== Top 5 LSTM Configurations with Lowest Loss =====")
for config, loss in sorted_lstm_losses:
    print(f"Optimizer: {config[0]}, Activation: {config[1]}, Window Size: {config[2]}, Num Cells: {config[3]}, Average Epochs: {average_epoch_lstm}, Loss: {loss}")
print("\n\n\n")
```

실행 후 loss별로 정렬한 후, 가장 loss가 적은 5개의 모델을 표시한다.

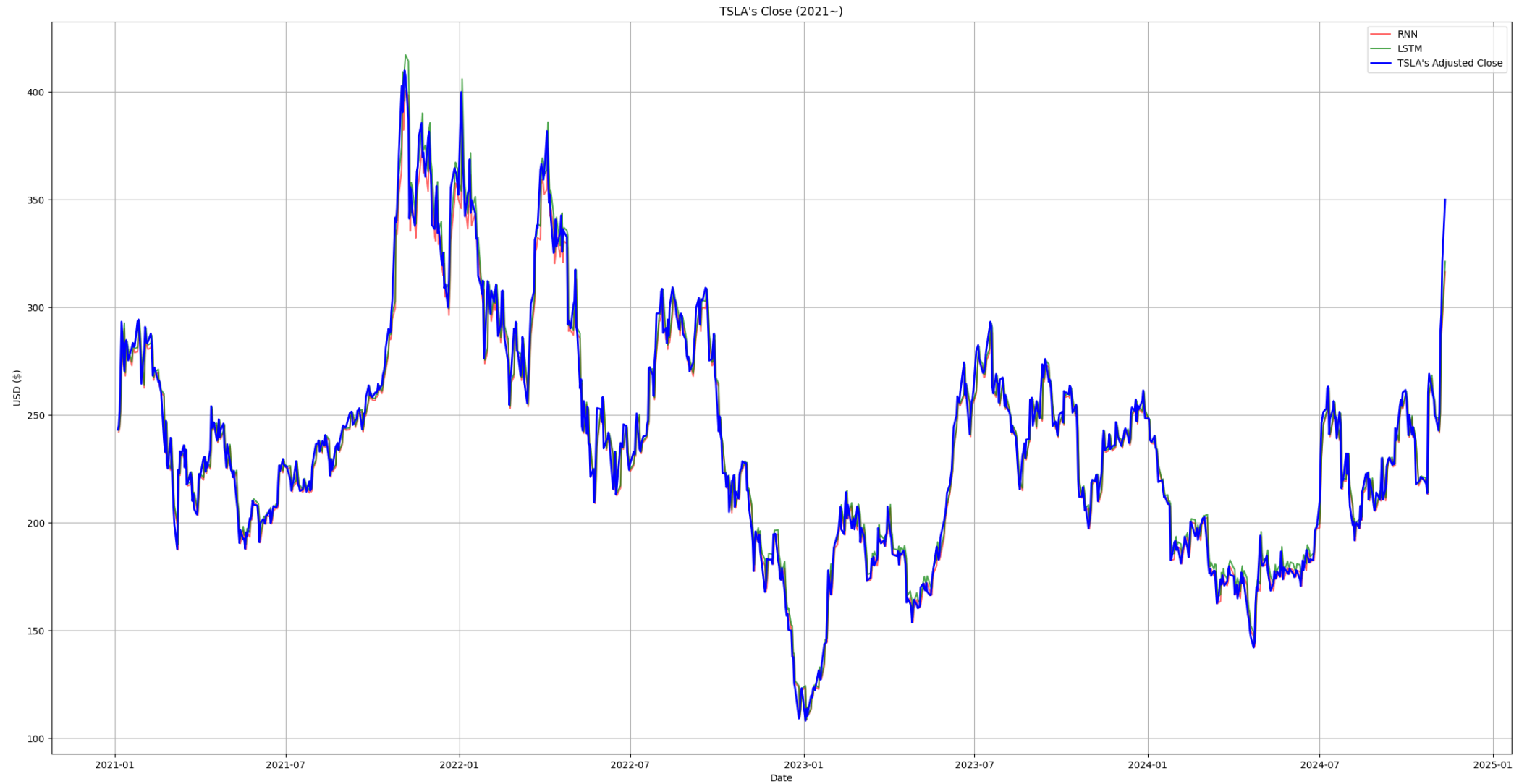
```
# ===== Top 5 RNN Configurations with Lowest Loss =====
# Optimizer: nadam, Activation: leaky_relu, Window Size: 1, Num Cells: 1, Average Epochs: 31.2, Loss: 5.577327829087153e-05
# Optimizer: adamax, Activation: leaky_relu, Window Size: 1, Num Cells: 1, Average Epochs: 31.2, Loss: 5.63266767130699e-05
# Optimizer: nadam, Activation: relu, Window Size: 1, Num Cells: 1, Average Epochs: 31.2, Loss: 5.673377017956227e-05
# Optimizer: adamax, Activation: selu, Window Size: 1, Num Cells: 1, Average Epochs: 31.2, Loss: 5.6737204431556165e-05
# Optimizer: adam, Activation: relu, Window Size: 1, Num Cells: 1, Average Epochs: 31.2, Loss: 5.72559583815746e-05

# ===== Top 5 LSTM Configurations with Lowest Loss =====
# Optimizer: adamax, Activation: relu, Window Size: 1, Num Cells: 1, Average Epochs: 44.8, Loss: 5.530421549337916e-05
# Optimizer: adamax, Activation: leaky_relu, Window Size: 1, Num Cells: 1, Average Epochs: 44.8, Loss: 5.568451888393611e-05
# Optimizer: nadam, Activation: leaky_relu, Window Size: 1, Num Cells: 1, Average Epochs: 44.8, Loss: 5.579991193371825e-05
# Optimizer: nadam, Activation: selu, Window Size: 1, Num Cells: 1, Average Epochs: 44.8, Loss: 5.586405313806608e-05
# Optimizer: adamax, Activation: selu, Window Size: 1, Num Cells: 1, Average Epochs: 44.8, Loss: 5.607605999102816e-05
```

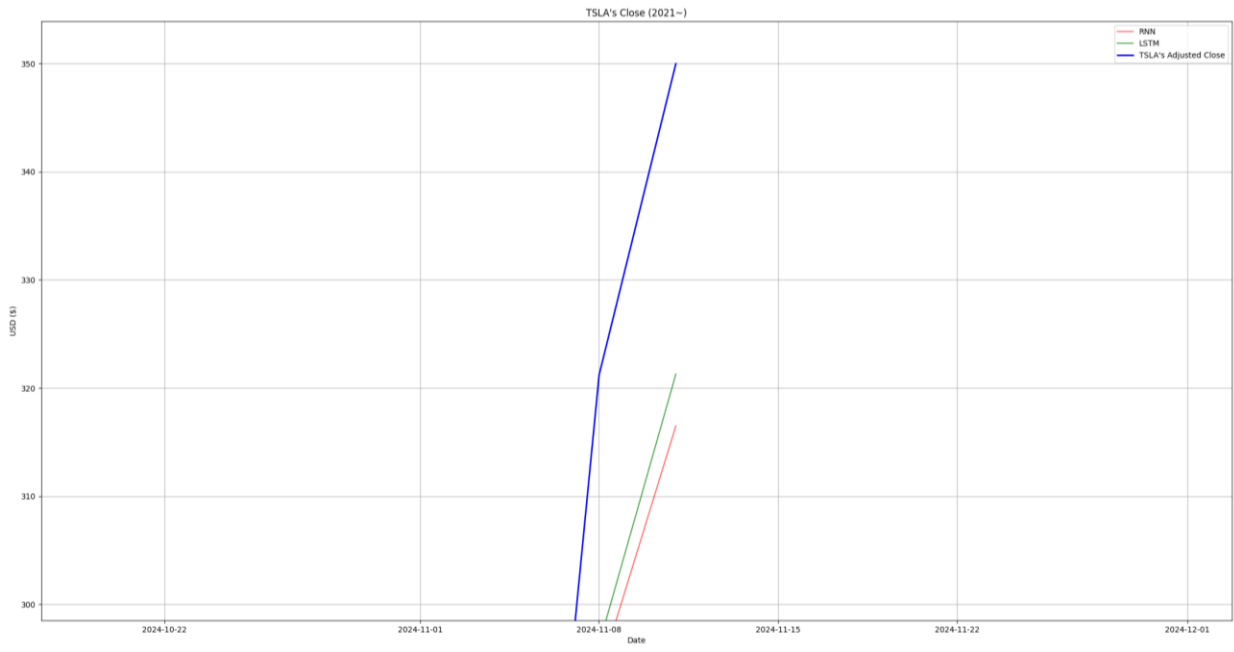
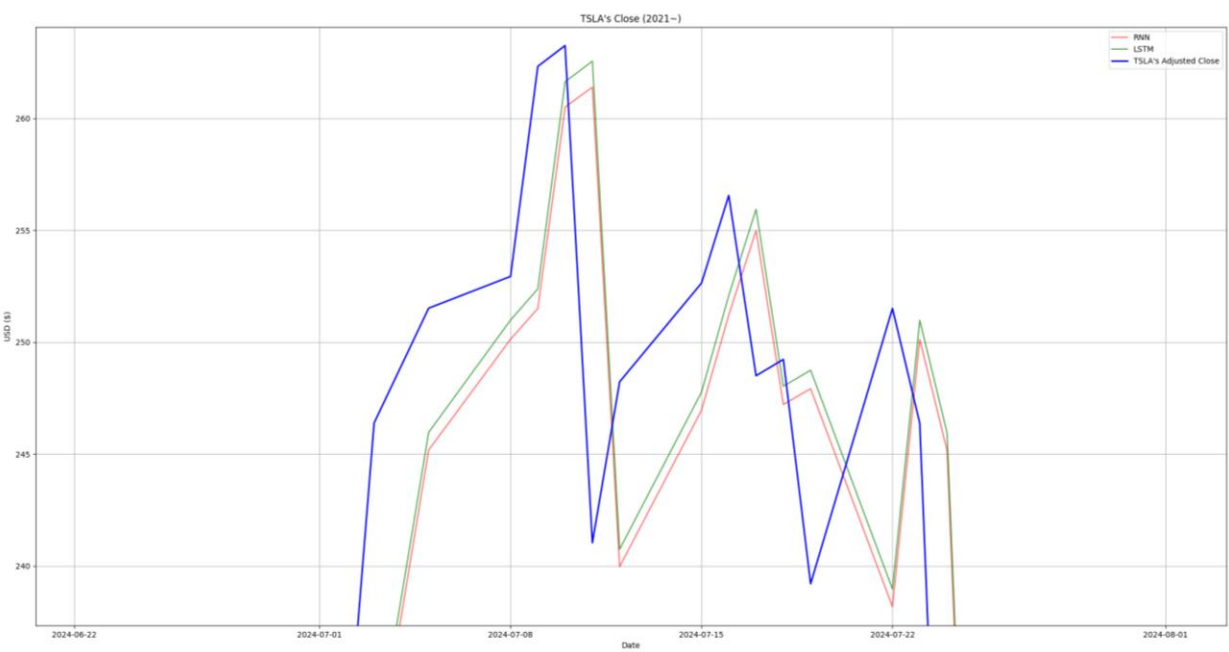
위는 실행 결과이며, 가장 높은 정확도를 가졌던 구조대로 모델을 구성하였다.

다만, LSTM은 활성화함수를 ReLU로 하니 ReLU의 특성상 음수 값을 0으로 만들기 때문에 특정 상황에서 출력이 0으로 고착되는 현상이 발견되어 두 번째로 정확도가 높았던 Leaky ReLU를 활성화함수로 사용하였다.

02 실습 #1 실행 결과



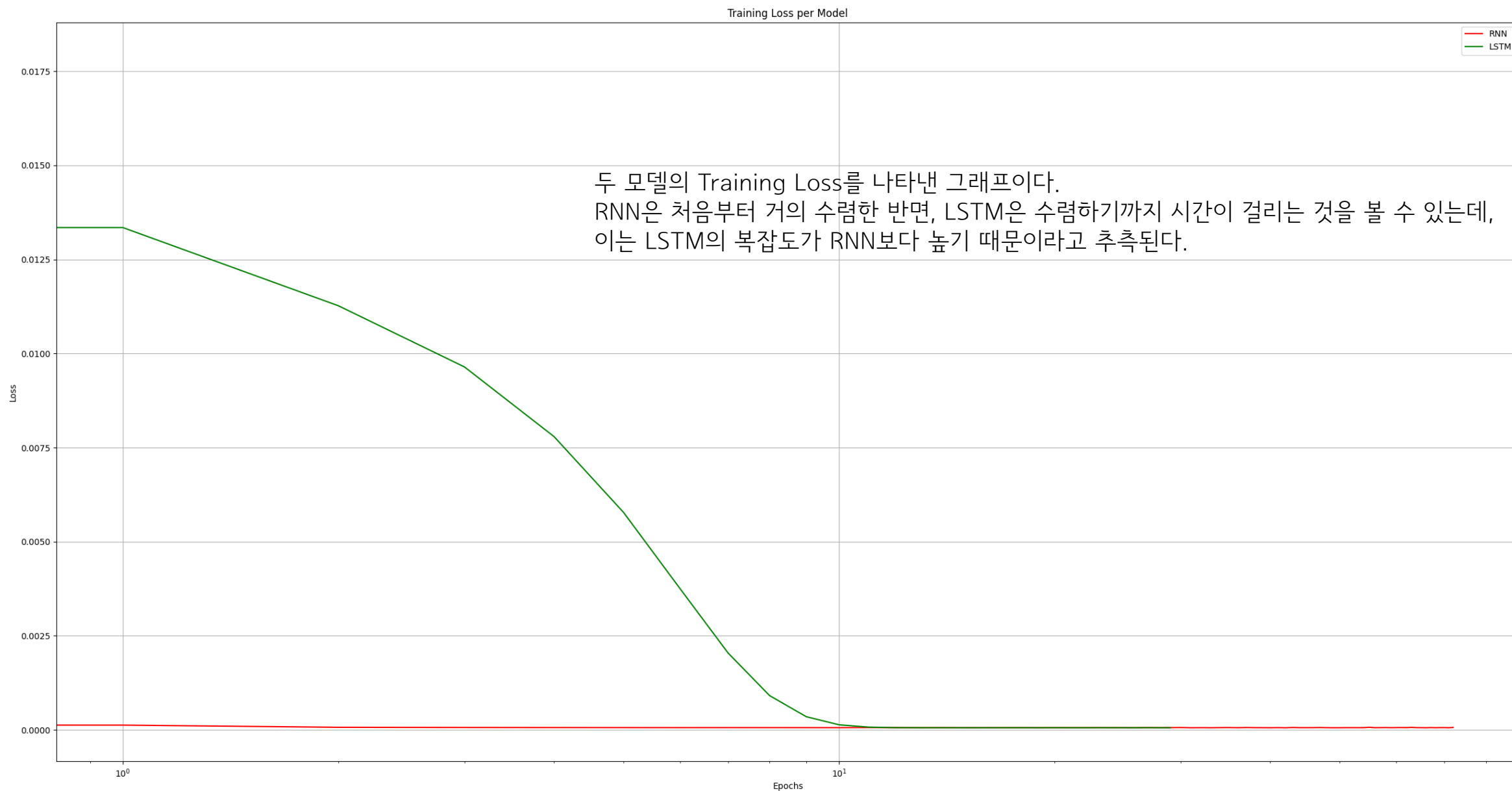
02 실습 #1 실행 결과



실행 결과를 확대한 사진이며, 정확하게 예측하고 있지는 못하는 모습이다.
특히, 맨 마지막 기간 최근 주가 폭등에 대해서는 잘 예측하지 못하고 있다.
이는 2024년 11월 10일 일요일에 예측된 수치이며, (11일 종가를 예측) 실제 11일의 종가는 350.00\$였다.

```
=====  
(RNN)      Predicted next day closing price: 343.77$  
(LSTM)      Predicted next day closing price: 351.58$  
=====
```

02 실습 #1 실행 결과



03

실습 #2

새로운 아이디어를 적용하여 주가 예측 모델의 한계점 개선

03 실습 #2 새로운 아이디어를 적용하여 주가 예측 모델의 한계점 개선

실습 #2) 새로운 아이디어를 적용하여 주가 예측 모델의 한계점 개선

- 실습 #1에서 사용한 데이터와 파라미터는 변경하지 말 것
- 실습 #1의 결과와 자신만의 새로운 아이디어를 적용한 순환 신경망의 결과 비교 분석

03 실습 #2 코드 분석

```
def build_gru_model(num_cells, activation, window_size):
    model = keras.models.Sequential()
    for i in range(num_cells):
        return_sequences = bool(i < num_cells - 1)
        if activation == 'leaky_relu':
            model.add(keras.layers.GRU(128, activation=keras.layers.LeakyReLU(alpha=0.01), return_sequences=return_sequences,
input_shape=(window_size, 1)))
        else:
            model.add(keras.layers.GRU(128, activation=activation, return_sequences=return_sequences, input_shape=(window_size, 1)))
    model.add(keras.layers.Dense(1, activation=activation if activation != 'leaky_relu' else keras.layers.LeakyReLU(alpha=0.01)))
    return model

def build_bidir_rnn_model(num_cells, activation, window_size):
    model = keras.models.Sequential()
    for i in range(num_cells):
        return_sequences = bool(i < num_cells - 1)
        if activation == 'leaky_relu':
            model.add(keras.layers.Bidirectional(keras.layers.SimpleRNN(128, return_sequences=return_sequences),input_shape=(window_size,
1)))
            model.add(keras.layers.LeakyReLU(alpha=0.01))
        else:
            model.add(keras.layers.Bidirectional(keras.layers.SimpleRNN(128, activation=activation,
return_sequences=return_sequences),input_shape=(window_size, 1)))
    model.add(keras.layers.Dense(1, activation=activation if activation != 'leaky_relu' else keras.layers.LeakyReLU(alpha=0.01)))
    return model
```

최적의 파라미터를 찾는 코드에 GRU와 Bi-Directional RNN을 추가하였다.

03 실습 #2 코드 분석

```
# ===== Top 5 RNN Configurations with Lowest Loss =====
# Optimizer: nadam, Activation: leaky_relu, Window Size: 1, Num Cells: 1, Average Epochs: 31.2, Loss: 5.577327829087153e-05
# Optimizer: adamax, Activation: leaky_relu, Window Size: 1, Num Cells: 1, Average Epochs: 31.2, Loss: 5.63266767130699e-05
# Optimizer: nadam, Activation: relu, Window Size: 1, Num Cells: 1, Average Epochs: 31.2, Loss: 5.673377017956227e-05
# Optimizer: adamax, Activation: selu, Window Size: 1, Num Cells: 1, Average Epochs: 31.2, Loss: 5.6737204431556165e-05
# Optimizer: adam, Activation: relu, Window Size: 1, Num Cells: 1, Average Epochs: 31.2, Loss: 5.72559583815746e-05

# ===== Top 5 LSTM Configurations with Lowest Loss =====
# Optimizer: adamax, Activation: relu, Window Size: 1, Num Cells: 1, Average Epochs: 44.8, Loss: 5.530421549337916e-05
# Optimizer: adamax, Activation: leaky_relu, Window Size: 1, Num Cells: 1, Average Epochs: 44.8, Loss: 5.568451888393611e-05
# Optimizer: nadam, Activation: leaky_relu, Window Size: 1, Num Cells: 1, Average Epochs: 44.8, Loss: 5.579991193371825e-05
# Optimizer: nadam, Activation: selu, Window Size: 1, Num Cells: 1, Average Epochs: 44.8, Loss: 5.586405313806608e-05
# Optimizer: adamax, Activation: selu, Window Size: 1, Num Cells: 1, Average Epochs: 44.8, Loss: 5.607605999102816e-05

# ===== Top 5 GRU Configurations with Lowest Loss =====
# Optimizer: adamax, Activation: selu, Window Size: 1, Num Cells: 1, Average Epochs: 20.8, Loss: 5.4472762712975964e-05
# Optimizer: adamax, Activation: leaky_relu, Window Size: 1, Num Cells: 1, Average Epochs: 20.8, Loss: 5.508438334800303e-05
# Optimizer: adamax, Activation: relu, Window Size: 1, Num Cells: 1, Average Epochs: 20.8, Loss: 5.6287757615791634e-05
# Optimizer: adamax, Activation: leaky_relu, Window Size: 1, Num Cells: 2, Average Epochs: 20.8, Loss: 5.733739089919254e-05
# Optimizer: nadam, Activation: selu, Window Size: 1, Num Cells: 1, Average Epochs: 20.8, Loss: 5.739692278439179e-05

# ===== Top 5 Bi-Directional RNN Configurations with Lowest Loss =====
# Optimizer: adamax, Activation: leaky_relu, Window Size: 1, Num Cells: 2, Average Epochs: 22.2, Loss: 5.821137165185064e-05
# Optimizer: nadam, Activation: leaky_relu, Window Size: 1, Num Cells: 1, Average Epochs: 22.2, Loss: 5.8720434026326984e-05
# Optimizer: adamax, Activation: leaky_relu, Window Size: 1, Num Cells: 1, Average Epochs: 22.2, Loss: 5.964088632026687e-05
# Optimizer: adam, Activation: leaky_relu, Window Size: 1, Num Cells: 1, Average Epochs: 22.2, Loss: 6.0010537708876655e-05
# Optimizer: adamax, Activation: leaky_relu, Window Size: 2, Num Cells: 1, Average Epochs: 22.2, Loss: 6.057571226847358e-05
```

해당 코드를 실행하고 나온 최적의 파라미터들이다.

03 실습 #2 코드 분석

```
BIDIRNN = keras.models.Sequential()  
BIDIRNN.add(keras.layers.Bidirectional(keras.layers.SimpleRNN(units=128, activation='linear', return_sequences=True,  
input_shape=(window_size, 1))))  
BIDIRNN.add(keras.layers.LeakyReLU(alpha=0.01))  
BIDIRNN.add(keras.layers.Bidirectional(keras.layers.SimpleRNN(units=128, activation='linear')))  
BIDIRNN.add(keras.layers.LeakyReLU(alpha=0.01))  
BIDIRNN.add(keras.layers.Dense(1, activation='linear'))  
BIDIRNN.compile(optimizer='adamax', loss='MSE')
```

Bi-Directional RNN을 구성하는 코드이다.

Leaky ReLU는 alpha라는 파라미터가 하나 더 있기에, activation='leaky_relu'로 하는 것이 아니라, 활성화함수를 우선 linear로 설정한 다음, 그 다음에 Leaky ReLU를 추가하였다.

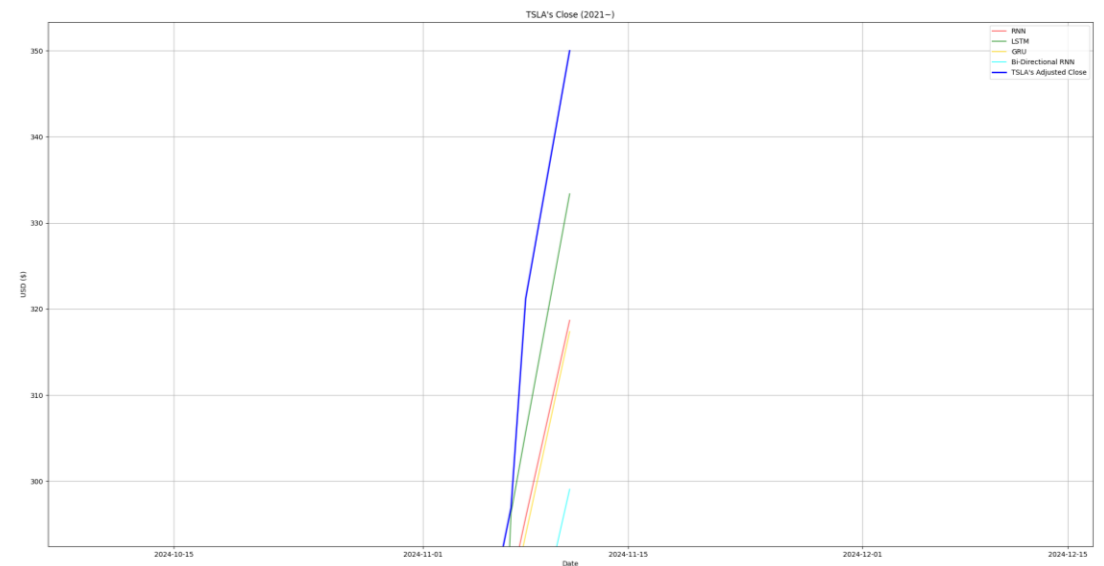
또한, keras의 구조 상 Bi-Directional 구조가 바로 있는것이 아니기 때문에 Vanilla RNN에 Bi-Directional 기능을 추가하는 방법으로 Bi-Directional RNN을 구현하였다.

최적의 파라미터에 의하면 Bi-Directional RNN은 셀이 2개일 때 최적인데, 그래서 셀을 2개로 설정한 다음 return_sequences도 True로 설정해 주었다.

03 실습 #2 실행 결과



03 실습 #2 실행 결과



각 모델들이 비슷한 정확도를 보이나, Bi-Directional RNN은 주가를 잘 예측하지 못하는 모습이다.

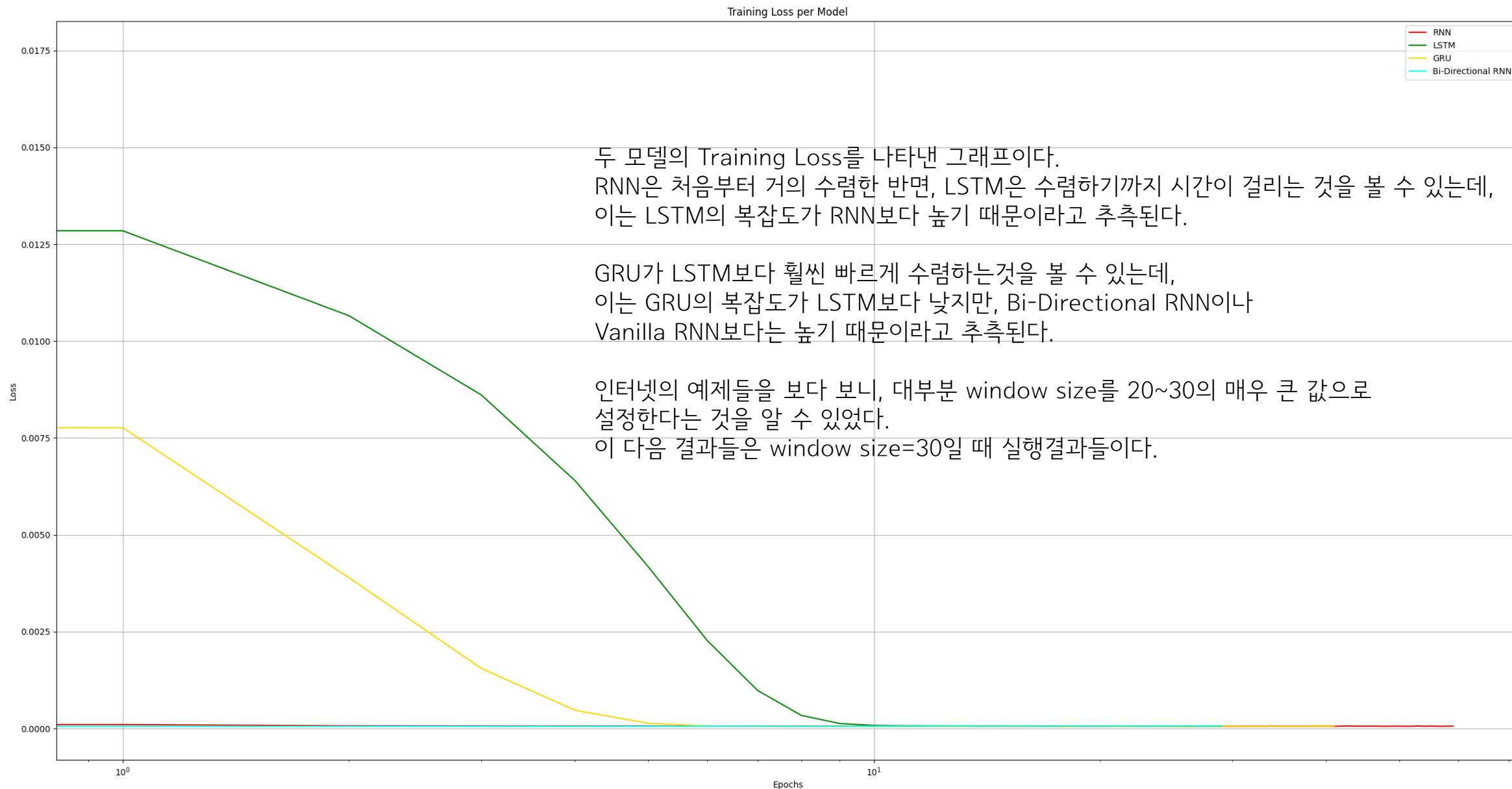
특히, 맨 마지막 기간 최근 주가 폭등 또한 새로운 모델들도 잘 예측하지 못하고 있다.

아래는 2024년 11월 10일 일요일에 예측된 수치이며, (11일 종가를 예측) 실제 11일의 종가는 350.00\$였다.

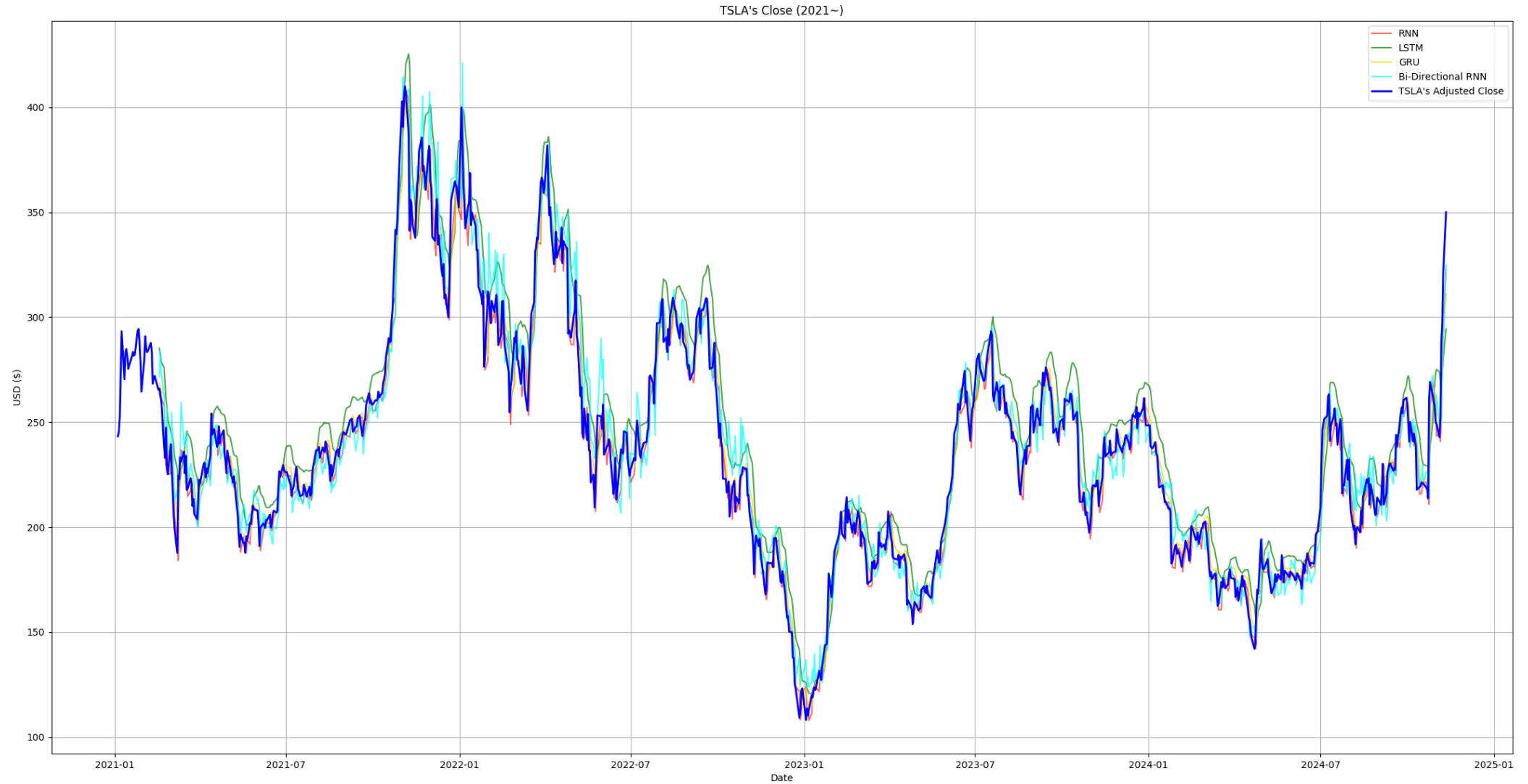
Bi-Directional RNN을 제외하고는 대체로 비슷하게 예측하는 것을 알 수 있으며, 몇 번의 실행 결과 LSTM과 GRU의 정확도가 가장 높은 경우가 많았다.

```
=====  
(RNN)      Predicted next day closing price: 345.80$  
(LSTM)      Predicted next day closing price: 366.91$  
(GRU)       Predicted next day closing price: 345.27$  
(Bi-Dir RNN) Predicted next day closing price: 325.69$  
=====
```

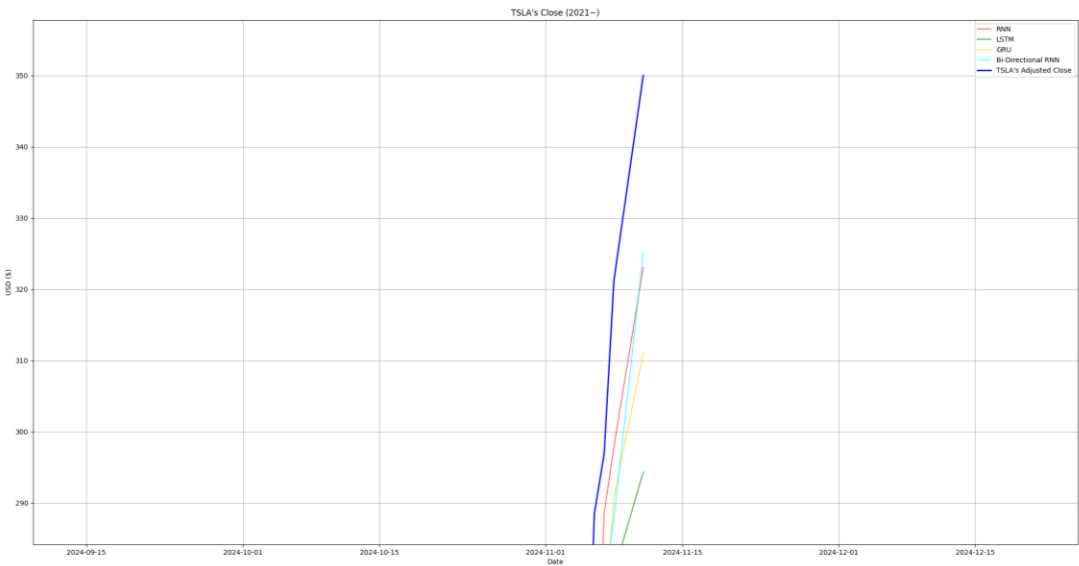
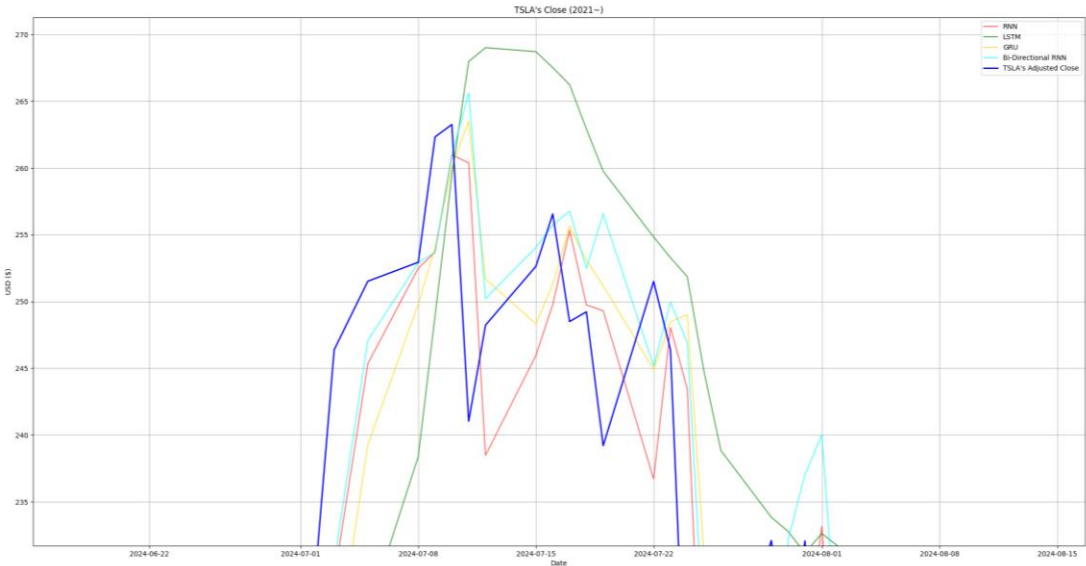
03 실습 #2 실행 결과



03 실습 #2 실행 결과



03 실습 #2 실행 결과

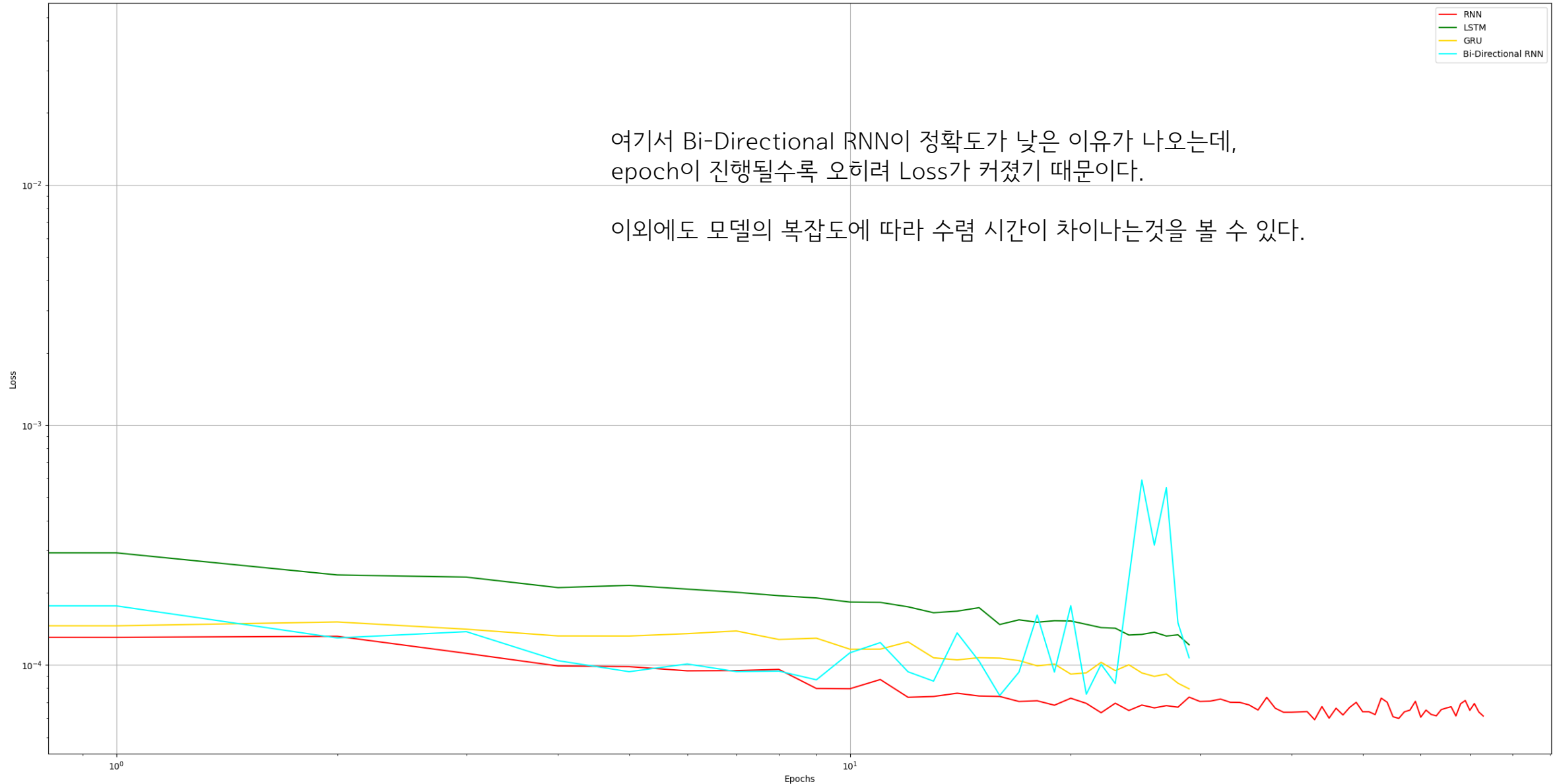


Window size를 30으로 늘리자 LSTM의 정확도가 크게 감소한 모습이다.
아래는 2024년 11월 10일 일요일에 예측된 수치이며, (11일 종가를 예측) 실제 11일의 종가는 350.00\$였다.
따라서 작은 Window size에서는 LSTM의 정확도가, 높은 Window size에서는 RNN 및 Bi-Directional RNN의 정확도가 높다고 할 수 있다.

```
=====  
(RNN)      Predicted next day closing price: 349.27$  
(LSTM)      Predicted next day closing price: 314.29$  
(GRU)       Predicted next day closing price: 336.28$  
(Bi-Dir RNN) Predicted next day closing price: 348.08$  
=====
```


03 실습 #2 실행 결과

Training Loss per Model



분석

주식 가격 예측이 정확하지 않은 이유

본 보고서에서 사용된 모델들은 RNN 계열 모델들로서, Sequence 데이터로 주식의 종가를 받아 그 다음 날의 가격을 예측하였다.

RNN 계열의 모델들은 각 데이터간의 순서가 있으며

그 순서 간의 연관성이 있을 것이라고 가정한 후 사용하는 네트워크이다.

실제로, 순서에 따른 연관성이 있는 “문장의 다음 단어 예측”같은 문제는 LSTM이나 다른 RNN들이 꽤나 잘하는 편이다.

그러나, 주식의 종가는 약간 다르다.

한 예로 이번 미국 대선에서 트럼프가 당선된 이후로 테슬라의 주가가 폭등하였는데, 이는 가격만으로는 알 수 없는 정보였다.

전날 나스닥의 증감률이나 연준의장의 발언 등등도 주가에 큰 영향을 미칠 것이다.

주식 가격은 기본적으로 Input이 너무 많은 데다가 랜덤성 또한 보이는 경향이 있다.

또한 효율적 시장 가설(Efficient Market Hypothesis, EMH)에 따르면,

시장에 공개된 모든 정보가 이미 주가에 반영되어 있기 때문에,

주가는 랜덤워크를 따른다고 가정한다.

즉, 미래 주가는 새로운 정보에 의해서만 결정되므로 예측이 매우 어렵다는 것인데,

이는 머신러닝에 의한 주가 예측을 더 어렵게 만든다.



고급머신러닝 보고서
감사합니다